

React JS Notes:

Introduction:

- React is JavaScript library created by Facebook.
- Most popular JavaScript library for creating UI.
- Also used by Netflix & Instagram.
- Used to create Single Page Applications (SPA)
- We can build modern, fast Single Page Applications or websites with React.

Is React JS a Library or a Framework?

- React is a Library, not a Framework.

What is a Library?

- A library in programming can be explained as a collection of codes. We use a library to write code in a much simpler way or to import a feature from it into our project. JQuery is a library for example.
- We can write JavaScript much simpler by using JQuery, or we can import written JQuery features to our project. The project itself is not dependent on a library.

What is a Framework?

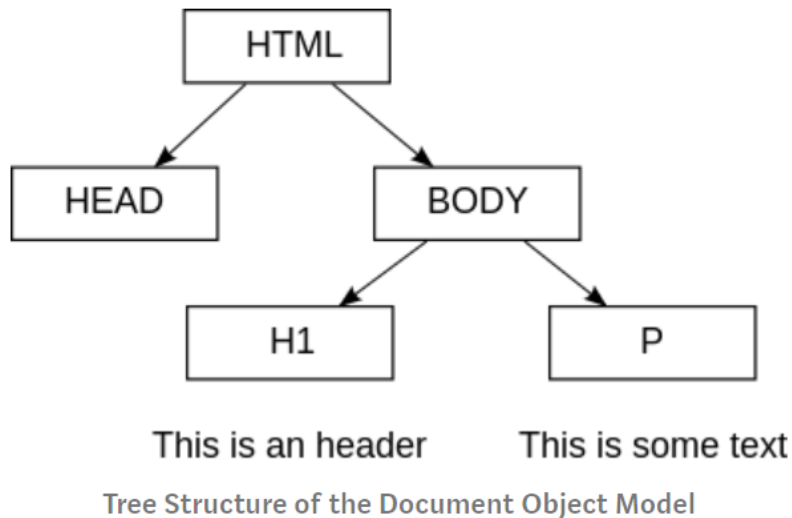
- A Framework, on the other hand, is a complete package of code with its own functionalities & libraries. A Framework has its own rules, you don't have much flexibility and the project is dependent on the Framework you use. Angular is an example of a framework.

How React actually works?

React Virtual DOM

- To understand the importance of React Virtual DOM, first, you need to know what DOM (Document Object Model) is.

- DOM is basically the representation of the HTML code in a webpage. The document is the webpage itself, the objects are the HTML tags. And finally, the model of DOM is a tree structure:



What is the benefit of Virtual DOM?

- Each time you make a change in the code, DOM will be completely updated and rewritten. This is an expensive operation and consumes lots of time. In this point, React provides a solution: The Virtual DOM.
- So when something changes:
 - React first creates an exact copy of the DOM
 - Then React figures out which part is new and only updates that specific part in the Virtual DOM
 - Finally, React copies only the new parts of the Virtual DOM to the actual DOM, rather than completely rewriting it.
- This approach makes a webpage much faster than a standard webpage. That's also one of the reasons why React is so popular.

React's Core Syntax: JSX

- In classic Frontend programming, we have separated HTML, CSS and JS file structures. React is a bit different. We don't have separated HTML files in React.

- In JSX syntax, we write HTML tags inside JavaScript.
- In React, for example, a simple JavaScript variable can be like this:
`const element = <h1>Hello!</h1>;`
- Normally, we can't assign an HTML tag to a JavaScript variable. But with JSX, we can. The code above you see is neither HTML nor JavaScript. It's an example of JSX.

So what is this JSX?

- JSX (JavaScript XML) is a syntax extension to JavaScript used by React. JSX is basically used to write HTML tags inside JavaScript. Later, the JSX code will be translated into normal JavaScript, by Babel.
- In summary, React doesn't have HTML files, HTML tags are rendered directly inside JavaScript. This approach makes React faster.

Do I have to work with JSX?

- You don't have to use JSX with React, but it is strongly recommended. JSX simplifies React and makes it easier to read. Let me give an example of React code with and without JSX.

React with JSX:

```
class Hello extends React.Component {  
  render() {  
    return <div>Hello {this.props.toWhat}</div>;  
  }  
}  
  
ReactDOM.render(  
  <Hello toWhat="World" />,  
  document.getElementById('root')  
);
```

React without JSX:

```
class Hello extends React.Component {
  render() {
    return React.createElement('div', null, `Hello
    ${this.props.toWhat}`);
  }
}

ReactDOM.render(
  React.createElement(Hello, {toWhat: 'World'}, null),
  document.getElementById('root')
);
```

Some important rules about JSX:

- We can't return more than one HTML element at once, but we can wrap the elements inside a parent HTML tag:

```
class Test extends React.Component {
  render() {
    return (
      <div>
        <p>Hello</p>
        <p>World</p>
      </div>
    );
  }
}
```

- We can use JSX inside for loops, if-else cases:

```
render() {
  if(condition==true) {
    return <p>This text</p>;
  } else {
    return <p>Another text</p>;
  }
}
```

- HTML attribute names like “class” becomes “className”.

```
<div className="myClass"></div>
```

- HTML tags must always be closed.

React Installation:

- React requires Nodejs.
- After installing Nodejs, open your Terminal or Command Prompt and type the following command to create your React app:

```
npx create-react-app my-app
```

```
cd my-app
```

- npx - It is a tool for executing node packages. [node package runner.]
x - executor.

What is a React Component?

- A component is an independent, reusable code block, which divides the UI into smaller pieces.
- In other words, we can think of components as LEGO blocks. Likewise we create a LEGO structure from many little LEGO blocks, we create a webpage or UI from many little code blocks (components).
- Smaller code blocks. Easy to maintain. They are reusable, easier to read, write & test.
- React has 2 types of components: Functional (Stateless) and Class (Stateful).

Functional (Stateless) Components:

- A functional component is basically a JavaScript (or ES6) function which returns a React element. According to React official docs, the function below is a valid React component:

```
function Welcome(props) {  
  return <h1>Hello, {props.name}</h1>;  
}
```

- This function is a valid React component because it accepts a single “props” (which stands for properties) object argument with data and returns a React element. — reactjs.org
- So we can define a React functional component as a JS Function:

```
function Example() {  
  return ( <h1>I'm a functional component!</h1> );  
};
```

- or as an ES6 arrow function:

```
const Example = () => {  
  return ( <h1>I'm a functional component!</h1> );  
};
```

- Both of the functions are valid React components. They may take props as an argument (when necessary), but they must return a React element.
- IMPORTANT: Functional components are also known as stateless components because, in the past, we couldn't do more complex things like React State (data) management or life-cycle methods in functional components.
- However, React introduced React Hooks in version 16.8, which now allows us to use state & other features in functional components.
- So a React Functional Component:
 - is a JavaScript / ES6 function
 - must return a React element
 - take props as parameter if necessary

Class (Stateful) Components

- Class components are ES6 classes. They are more complex than functional components including constructors, life-cycle methods, render() function and state (data) management.

- In the example below, we can see how a simple class component looks like:

```
import React, { Component } from 'react';

class ExampleComponent extends Component {
  render() {
    return (
      <div>This is an example component.</div>
    );
  }
}

export default ExampleComponent;
```

- Here, the ExampleComponent class extends Component, so React understands that this class is a component, and it renders (returns) a React Element.
- So, a React class component:
 - is an ES6 class, will be a component once it 'extends' React component.
 - can accept props (in the constructor) if needed
 - can maintain its own data with state
 - must have a render() method which returns a React element (JSX), or null

How to call a component?

- A component is being called like an HTML tag, but starting with a capital letter:

`<ExampleComponent />`

- Components are the core of React. Having a better knowledge of when and how to use functional & class components not only makes your React app better performance, readable and testable, but also makes you a better programmer.

What is “Props” and how to use it in React?

- React has a different approach to data flow & manipulation than other frameworks.

What is Props?

- React is a component-based library which divides the UI into little reusable pieces. In some cases, those components need to communicate (send data to each other) and the way to pass data between components is by using props.
- “Props” is a special keyword in React, which stands for properties and is being used for passing data from one component to another.
- But the important part here is that data with props are being passed in a uni-directional flow. (one way from parent to child)
- Furthermore, props data is read-only, which means that data coming from the parent should not be changed by child components.

Using Props in React

- Firstly, define an attribute and its value(data)
- Then pass it to child component(s) by using Props
- Finally, render the Props Data

```
class ParentComponent extends Component {  
  render() {  
    return (  
      <h1>  
        I'm the parent component.  
        <ChildComponent />  
      </h1>  
    );  
  }  
}
```

```
const ChildComponent = () => {  
  return <p>I'm the 1st child!</p>;  
};
```


- The problem here is that, when we call the ChildComponent multiple times:

```
class ParentComponent extends Component {  
  render() {  
    return (  
      <h1>  
        I'm the parent component.  
        <ChildComponent />  
        <ChildComponent />  
        <ChildComponent />  
      </h1>  
    );  
  }  
}
```

- It always renders the same string again and again:

I'm the parent component.

I'm the 1st child

I'm the 1st child

I'm the 1st child

- But what we like to do here is to get dynamic outputs, because each child component may have different data and let's see how we can solve this issue by using props...
- 1st Step: Defining Attribute & Data
- We already know that we can assign attributes and values to HTML tags:

```
<a href="www.google.com">Click here to visit Google</a>;
```

- Likewise, we can do the same for React components. We can define our own attributes & assign values with interpolation { }:

```
<ChildComponent someAttribute={value} anotherAttribute={value}/>
```

- Let's declare a "text" attribute to the ChildComponent and then assign a string value: "I'm the 1st child".

```
<ChildComponent text={"I'm the 1st child"} />
```

- Now the ChildComponent has a property and a value. Next, we need to pass it via Props.
- 2nd Step: Passing Data using Props
- OK, now let's take the "I'm the 1st child!" string and pass it by using props.
- Passing props is very simple. Like we pass arguments to a function, we pass props into a React component and props bring all the necessary data.
- Arguments passed to a function:

```
const addition = (firstNum, secondNum) => {
  return firstNum + secondNum;
};
```

- Arguments passed to a React component:

```
const ChildComponent = (props) => {
  return <p>I'm the 1st child!</p>;
};
```

- Props are arguments passed into React components.
- Final Step: Rendering Props Data
- Prop is an Object
- we will render the props object by using string interpolation:
- {props}
- Log props to the console.
- console.log(props);

► Object { text: "I'm the 1st child" }

- As we can see, Props returns back an object. In JavaScript, we can access to object elements with dot(.) notation. So, let's render our text property with interpolation:

```
const ChildComponent = (props) => {  
  return <p>{props.text}</p>;  
};
```

I'm the parent component.

I'm the 1st child

- And that's it! We've achieved to render the data coming from the parent component.
- let's do the same for other child components:

```
class ParentComponent extends Component {  
  render() {  
    return (  
      <h1>  
        I'm the parent component.  
        <ChildComponent text={"I'm the 1st child"} />  
        <ChildComponent text={"I'm the 2nd child"} />  
        <ChildComponent text={"I'm the 3rd child"} />  
      </h1>  
    );  
  }  
}
```

I'm the parent component.

I'm the 1st child

I'm the 2nd child

I'm the 3rd child

- As we can see, each ChildComponent renders now its own prop data. So this is how we can use Props for passing data and converting static components into dynamic ones.

Recap

- Props stand for properties and is a special keyword in React
- Props are being passed to components like function arguments
- Props can only be passed to components in one-way (parent to child)
- Props data is immutable (read-only)

Understanding State:

- Props are only being used for passing data. They are read-only which means that components receiving data by props are not able to change it.
- However, in some cases, a component may need to manipulate data and that's not possible with props.
- So React provides another feature for data manipulation which is known as State.

What is State?

- State is a special object that holds dynamic data, which means that state can change over time and anytime based on user actions or certain events.
- State is private and belongs only to its component where defined, cannot be accessed from outside, but can be passed to child components via props.
- State is initialized inside its component's constructor method.

- When a change in the state is made, state shouldn't be modified directly. Instead, state updates should be made with a special method called `setState( )`.
- State should not be overly-used in order to prevent performance problems.

Using State in a Component

- In the earlier days, state could only be used in class components, but after the introduction of React Hooks, state now can be used both in class & functional components.

Creating the State

- A class has a special method called `constructor( )` and it is being called during object creation. We can also initialize our object properties.
- The same rule applies to state. Since state is also an object, it should be initialized inside the constructor method:

```
constructor() {  
  this.state = {  
    id: 1,  
    name: "test"  
  };  
}
```

- and later we can render the properties of the state object with JavaScript's dot notation, inside the `render ( )` method:

```

class Test extends React.Component {

  constructor() {
    this.state = {
      id: 1,
      name: "test"
    };
  }

  render() {
    return (
      <div>
        <p>{this.state.id}</p>
        <p>{this.state.name}</p>
      </div>
    );
  }
}

```

Updating the State

- A Component's state can change under some circumstances like a server response or user interaction (clicking on a button, scrolling the page etc).
- So when data changes, when a change in the state happens, React takes this information and updates the UI.
- The important point here is that we should not modify the state directly.
- [Do Not Modify State Directly – React Official Docs](#)

```

this.state.name = "testing state"; // wrong

```

Using setState()

- Below you can see the right way of state changes in React:

```

this.setState({
  name: "testing state"
});

```

- The reason why we should use `setState( )` is that because it's the only way to notify React for data changes. Otherwise React won't be notified and won't be able to update the UI.

Array.map() method

- The `map()` method is one of the most useful and often used.
- It calls the function for each element of the array and returns the array of results.
- The `map()` method calls the provided function once for each element in an array, in order.
- **Note:** `map()` does not execute the function for array elements without values.
- **Note:** this method does not change the original array.

Syntax

```
array.map(function(currentValue, index, arr))
```

Parameter Values

Parameter	Description								
<i>function(currentValue, index, arr)</i>	Required. A function to be run for each element in the array. Function arguments: <table> <tr> <th>Argument</th><th>Description</th></tr> <tr> <td><i>currentValue</i></td><td>Required. The value of the current element</td></tr> <tr> <td><i>index</i></td><td>Optional. The array index of the current element</td></tr> <tr> <td><i>arr</i></td><td>Optional. The array object the current element belongs to</td></tr> </table>	Argument	Description	<i>currentValue</i>	Required. The value of the current element	<i>index</i>	Optional. The array index of the current element	<i>arr</i>	Optional. The array object the current element belongs to
Argument	Description								
<i>currentValue</i>	Required. The value of the current element								
<i>index</i>	Optional. The array index of the current element								
<i>arr</i>	Optional. The array object the current element belongs to								

Return Value: An Array containing the results of calling the provided function for each element in the original array.

```
let lengths = ["Bilbo", "Gandalf", "Nazgul"].map(item => item.length);
console.log(lengths); // 5,7,6
```

Array.filter() method

- The filter() method creates an array filled with all array elements that pass a test (provided as a function).
- Note: filter() does not execute the function for array elements without values.
- Note: filter() does not change the original array.

Syntax

```
array.filter(function(currentValue, index, arr))
```

Parameter Values

Parameter	Description								
<i>function(currentValue, index, arr)</i>	Required. A function to be run for each element in the array. Function arguments: <table><tr><th>Argument</th><th>Description</th></tr><tr><td><i>currentValue</i></td><td>Required. The value of the current element</td></tr><tr><td><i>index</i></td><td>Optional. The array index of the current element</td></tr><tr><td><i>arr</i></td><td>Optional. The array object the current element belongs to</td></tr></table>	Argument	Description	<i>currentValue</i>	Required. The value of the current element	<i>index</i>	Optional. The array index of the current element	<i>arr</i>	Optional. The array object the current element belongs to
Argument	Description								
<i>currentValue</i>	Required. The value of the current element								
<i>index</i>	Optional. The array index of the current element								
<i>arr</i>	Optional. The array object the current element belongs to								

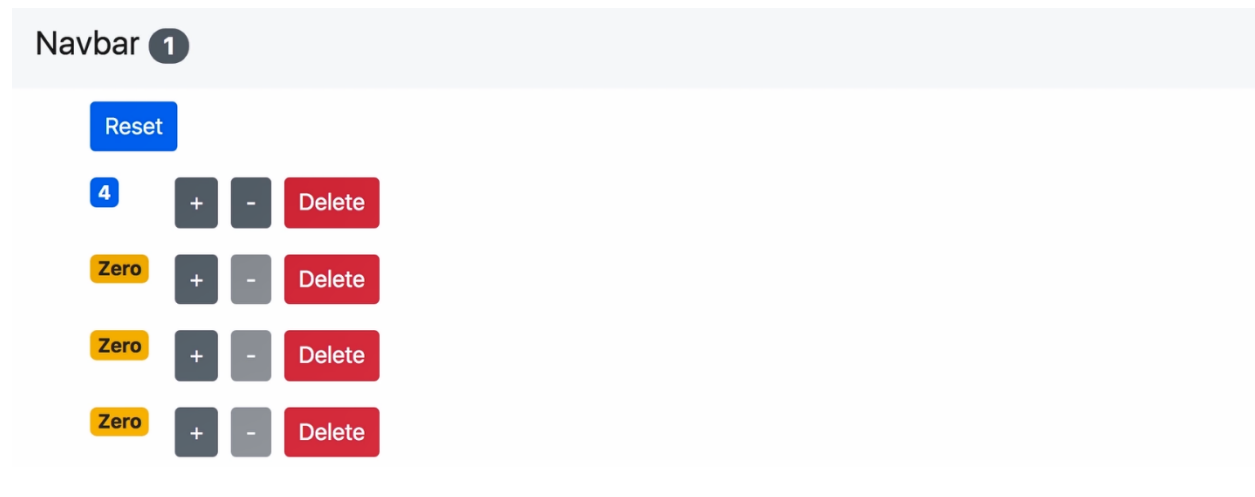
Return Value:	An Array containing all the array elements that pass the test. If no elements pass the test it returns an empty array.
----------------------	--

```
let users = [  
  {id: 1, name: "John"},  
  {id: 2, name: "Pete"},  
  {id: 3, name: "Mary"}  
];
```



```
// returns array of the first two users  
let someUsers = users.filter(item => item.id < 3);  
console.log(someUsers.length); // 2
```

Simple App that we want to build using above concepts:



Lifecycle methods:

- Every component of React application goes through some phases during the life cycle.
- There are main 3 lifecycle phases of components:
 - Mount phase
 - Update phase
 - Unmount phase

Mount Phase:

- This is a phase where component instance is created and inserted into the DOM.
- Mount phase has 3 life cycle methods: constructor, render, componentDidMount. React will call these methods in the same sequence.

componentDidMount()

- Whenever this method is called, React has already rendered our component and put it into the DOM. Therefore, if there is any initialization you want to perform that relies on the DOM, do it here and now.
- State: You can set the state with `this.setState()`. Whenever you do this, it will also trigger a re-render of the component.
- Use Cases: You can use `componentDidMount` to fetch data from a server with AJAX calls. We can add event listeners inside `componentDidMount`.

Update Phase:

- If props or state of a component are changed for whatever reason, an update of the component is performed. However, this means that the component has to be re-rendered.
- In this phase, we have 2 lifecycle methods, `render` and `componentDidUpdate`.

Routing:

What is react router?

React router is a routing library built on top of the react which is used to create the routing in react apps.

How to install the react router?

To install the react router you need to download the react-router-dom package by running the following commands.

```
npm install react-router-dom
```

- React router gives us three components [Route,Link,BrowserRouter] which help us to implement the routing.
- In the Route component, we need to pass the two props
 - path: it means we need to specify the path.
 - component: which component user needs to see when they will navigate to that path.

What is a 404 page?

A 404 page is also called not found page it means when a user navigates to the wrong path that doesn't present in the website we need to show the not found page.

How to add a 404 page in react?

We need to import another component called Switch which is provided by the react router.

What is Switch?

<Switch> returns only one first matching route.

Switch component helps us to render the components only when path matches otherwise it fallbacks to the not found component.

URL Parameters

URL parameters helps us to render the same component based on its dynamic url.

```
<Route path="users/:id" component={Users} />
```

- How to access route parameters:

```
this.props.match.params.id
```

NavLink:

It is used to style the active routes so that user knows on which page he or she is currently browsing on the website.

What is the difference between NavLink and Link?

The link is used to navigate the different routes on the site. But NavLink is used to add the style attributes to the active routes.

Programmatically navigate

What is Programmatic navigation?

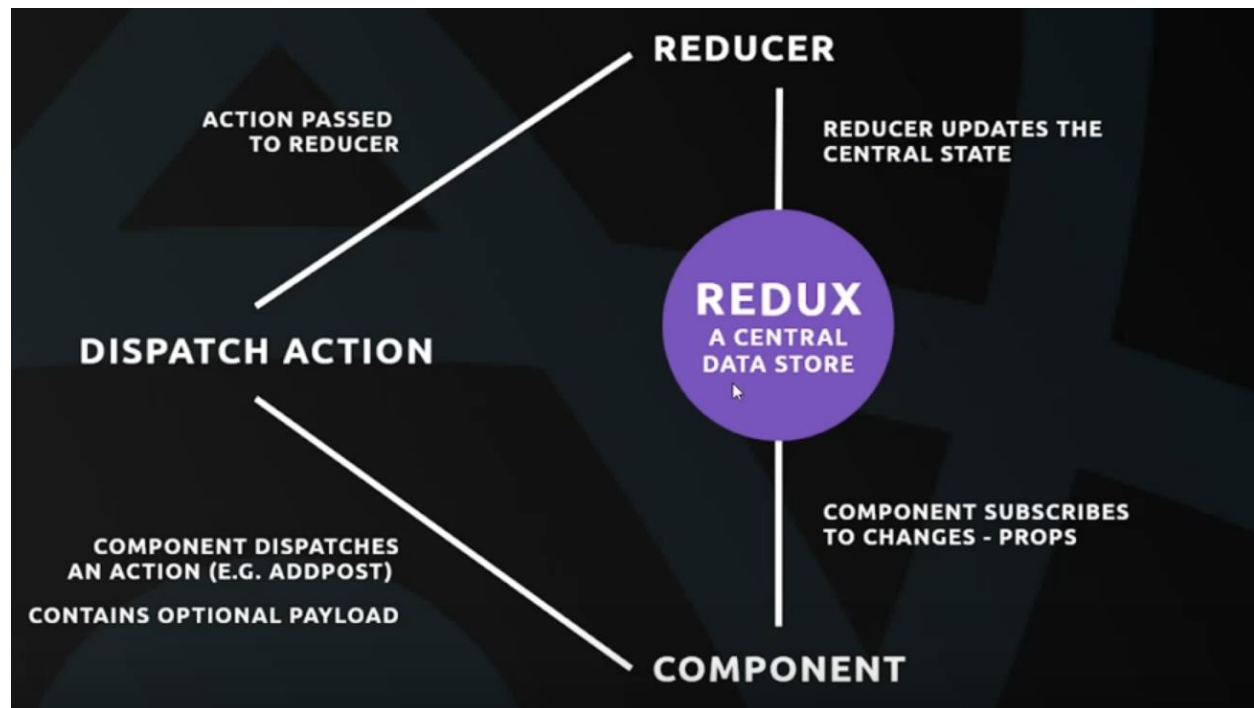
It means we need to redirect the user when an event happens on that route.

For example, when a user is successfully logged in he or she will be redirected to the home page.

How to navigate programmatically in react-router?

To navigate programmatically we need to take the help of history object which is passed by the react-router.

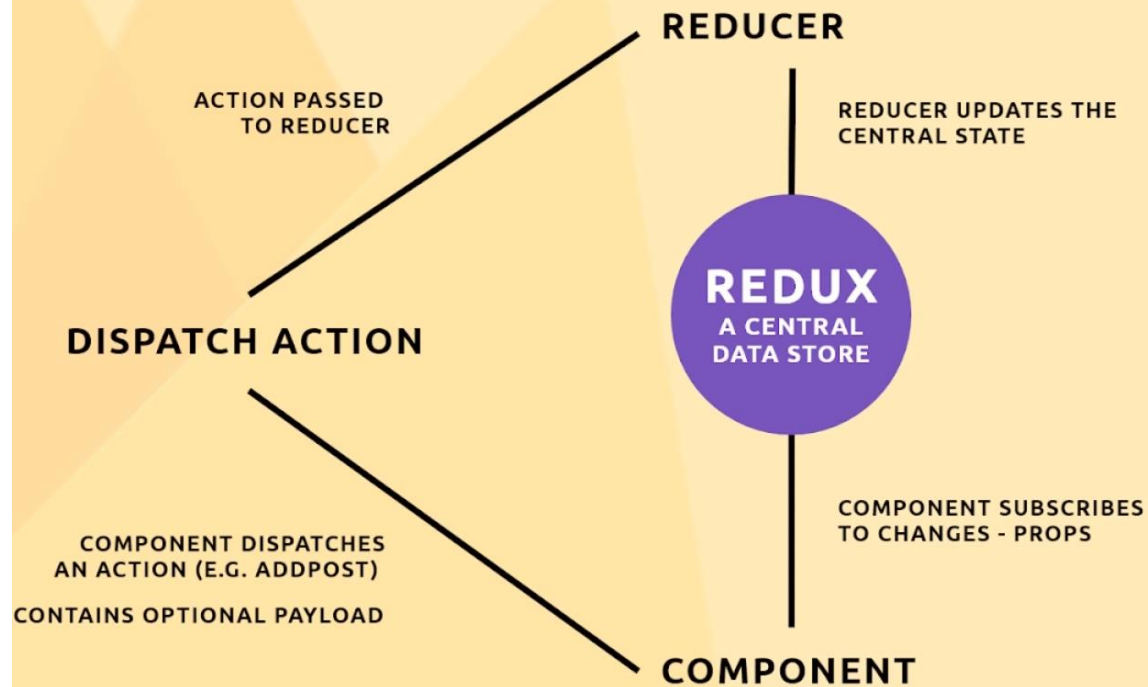
Redux:

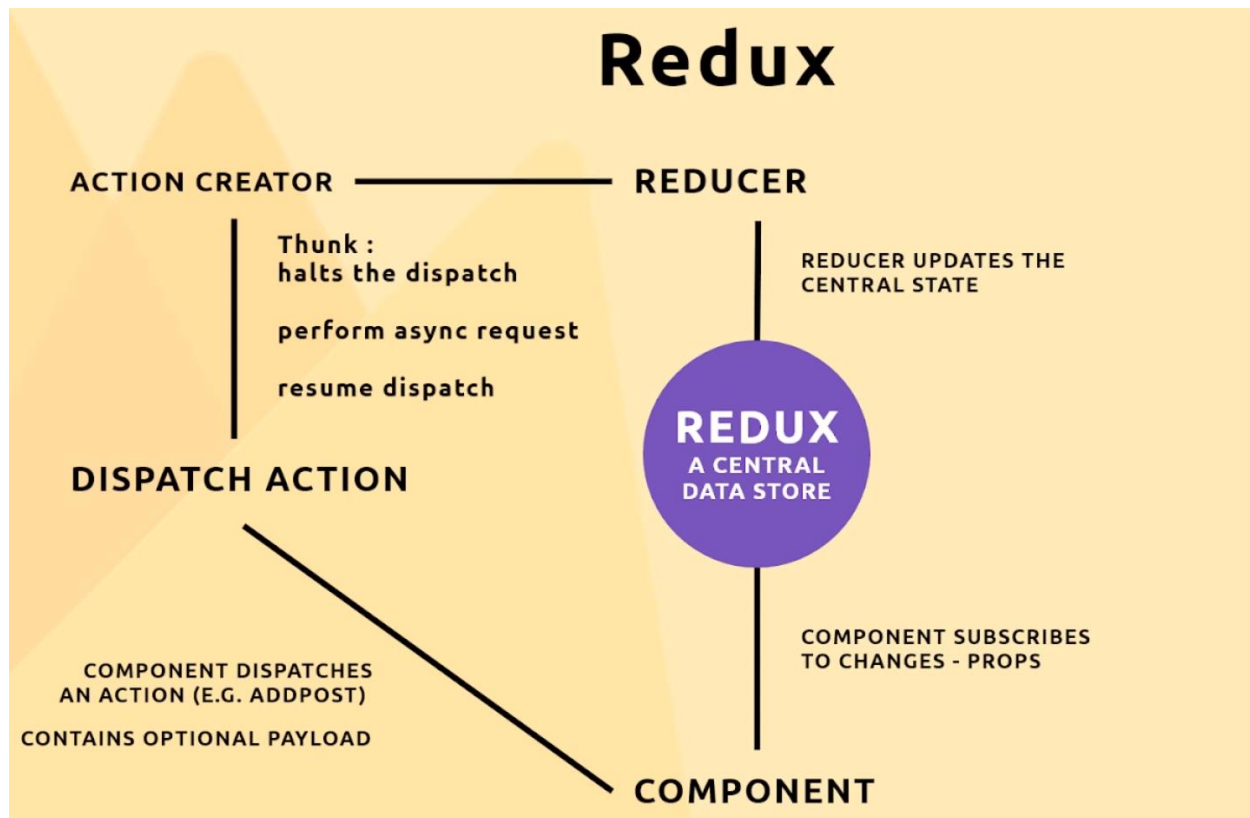


Create Post

All Posts

Redux





Connect(mapStateToProps, mapDispatchToProps).

- 1) Create mapDispatchToProps.
- 2) It is function that accepts dispatch as an argument and returns a function which is responsible for dispatching an action.
- 3) Dispatch function is getting action from action creator.
- 4) Action Creator: It is a callback function that returns an action.
It is a place where we have to make async calls and once the operation is done, we have to continue dispatching an action.